北京理工大学

本科生毕业设计（论文）

LeRobot 平台感知—推理—执行流程的系统级性能剖析研究

System-Level Performance Profiling of Perception-Inference-Execution Pipeline in the LeRobot Framework

学　　院：　计算机学院

专　　业：　软件工程

班　　级：　08012204

学 生 姓 名：　俞乐楠

学　　号：　1120221303

指 导 教 师：　王勇

20 26 年 4 月 20 日

原创性声明

本人郑重声明：所呈交的毕业设计（论文），是本人在指导老师的指导下独立进行研究所取得的成果。除文中已经注明引用的内容外，本文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。

特此申明。

本人签名：_____ 日 期：_____ 年 ____ 月 ____ 日

关于使用授权的声明

本人完全了解北京理工大学有关保管、使用毕业设计（论文）的规定，其中包括：学校有权保管、并向有关部门送交本毕业设计（论文）的原件与复印件；学校可以采用影印、缩印或其它复制手段复制并保存本毕业设计（论文）；学校可允许本毕业设计（论文）被查阅或借阅；学校可以学术交流为目的，复制赠送和交换本毕业设计（论文）；学校可以公布本毕业设计（论文）的全部或部分内容。

本人签名：_____ 日 期：_____ 年 ____ 月 ____ 日

指导教师签名：_____ 日 期：_____ 年 ____ 月 ____ 日

LeRobot 平台感知—推理—执行流程的系统级性能剖析研究

摘　要

[TODO：正式摘要待实验完成后撰写，约 300-500 字。]

本文以树莓派 5（ARM Cortex-A76，无 GPU）为目标平台，对 LeRobot 框架下 ACT 机器人学习策略的感知—推理—执行三阶段时序特征进行系统级剖析。通过在 Python 应用层、PyTorch 运行时层、Linux 内核层及硬件 I/O 层的多工具联合测量（Python logging / torch.profiler / ftrace / strace），端到端量化了各阶段的时间开销，定位了主要性能瓶颈，并从算法、运行时、系统架构三个维度梳理了可行的优化路径。

实验结果表明，ACT 单次推理耗时 2000–4000 ms，占总循环时间的 99% 以上，是系统主瓶颈；观测和执行阶段开销极低（各约 5 ms），系统调用本身的开销可忽略不计，排除了 I/O 路径为主要瓶颈的假设。在 chunk_size=100 配置下，系统实际帧率约 18–19 fps，距 30 fps 目标仍有明显差距，瓶颈根源在于 ARM CPU 矩阵乘法（GEMM）计算能力不足，而非感知复杂度或任务类型。

关键词： LeRobot；ACT；性能剖析；ARM 推理；chunk_size；感知-推理-执行

System-Level Performance Profiling of Perception-Inference-Execution Pipeline in the LeRobot Framework

Abstract

[TODO: English abstract to be finalized after experiments complete, approximately 300 words.]

This paper presents a system-level performance analysis of the ACT (Action Chunking with Transformers) policy running on a Raspberry Pi 5 (ARM Cortex-A76, no GPU) within the LeRobot framework. By combining multi-layer profiling tools across the Python application layer, PyTorch runtime layer, Linux kernel layer, and hardware I/O layer (Python logging, torch.profiler, ftrace, strace), we quantify the end-to-end time distribution across perception, inference, and execution stages, identify the primary bottleneck, and systematically evaluate feasible optimization paths.

Results show that a single ACT inference takes 2000–4000 ms, accounting for over 99% of the total loop time, making it the dominant bottleneck. Perception and execution stages contribute only ≈5 ms each; system call overhead is negligible, ruling out I/O as a bottleneck. At chunk_size=100, the system achieves ≈18–19 fps against a 30 fps target. The root cause is insufficient ARM CPU matrix-multiplication (GEMM) throughput, independent of task visual complexity or object count.

Key Words: LeRobot; ACT; performance profiling; ARM inference; chunk_size; perception-inference-execution

目录

第一章 绪论 **1**

1.1 研究背景与意义 1

1.2 研究目标 2

1.3 主要贡献 2

第二章 相关工作 **2**

2.1 机器人操作系统与通信框架 2

2.2 开源机器人硬件平台 3

2.3 机器人策略模型 3

2.4 嵌入式边缘 AI 推理 4

2.5 系统级 Profiling 技术 4

第三章 系统架构与负载特征分析 **5**

3.1 LeRobot 软件栈架构 5

3.2 三类典型工作负载特征 5

3.3 感知—推理—执行三阶段划分 6

第四章 跨层次性能分析模型与剖析方法 **6**

4.1 四层性能分析模型 6

4.2 工具链选型 6

4.3 实验配置 7

第五章 感知阶段性能剖析 **7**

5.1 时间剖析 7

5.1.1 摄像头路径 7

5.1.2 舵机路径 8

5.2 CallStack 源码分析 8

5.2.1 摄像头路径调用链 8

5.2.2 舵机路径调用链 8

第六章 推理阶段性能剖析 **9**

6.1 时间剖析 9

6.1.1 chunk_size 参数扫描 9
6.1.2 三类任务推理时间对比 9
6.1.3 算子级耗时分解 10
6.1.4 PyTorch 线程行为 10
6.2 CallStack 源码分析 10
6.2.1 ACT 推理完整调用链 10

第七章 执行阶段性能剖析 11
7.1 时间剖析 11
7.1.1 sync_write 路径 11
7.2 CallStack 源码分析 12
7.2.1 舵机写路径完整调用链 12

第八章 异步化可行性调研 12
8.1 摄像头异步读 12
8.2 推理异步 12
8.3 舵机异步读 13
8.4 舵机异步写 13

第九章 性能瓶颈分析与优化建议 14
9.1 瓶颈归因 14
9.2 跨阶段综合分析 14
9.2.1 时间预算表（单帧，33.3 ms 目标） 14
9.2.2 系统调用分布（40 s episode） 14
9.3 优化方向分析 15
9.4 软件架构改进方向 15

第十章 总结与展望 15
10.1 主要工作总结 15
10.2 研究局限性 16
10.3 未来工作 16

参考文献 17

附录 **18**

致谢 **19**

第一章　绪论

1.1　研究背景与意义

近年来，协作机器人在工业制造、医疗服务、仓储物流等领域加速落地，对自主操作能力的需求持续增长。以 RT-2[1]、OpenVLA[2] 为代表的视觉—语言—动作（Vision-Language-Action, VLA）大模型，将视觉感知与动作决策统一到端到端网络中，显著提升了机器人在开放场景下的泛化能力。然而，此类模型的参数量和计算量远超传统控制方法，如何在资源受限的嵌入式平台上实现实时推理，成为制约其部署的关键瓶颈。

在开源机器人学习框架方面，HuggingFace 推出的 LeRobot[3] 集成了 ACT（Action Chunking with Transformers)[4]、Diffusion Policy[5] 等模仿学习策略，并提供从数据采集、模型训练到在线推理的完整工具链；同时配套推出 SO-101 开源六自由度机械臂，实现了软件框架与低成本硬件平台的一体化开源。Physical Intelligence 提出的 π_0[6] 以流匹配为动作生成核心，在双臂协作任务上表现出色；斯坦福 Mobile ALOHA[7] 以低成本仿生平台搭配联合学习，实现了全身遥操作控制。LeRobot 的在线推理循环采用”感知—推理—执行”三阶段紧耦合架构：每帧依次完成传感器观测采集（`get_observation()`）、策略网络推理（`select_action()`）和舵机指令下发（`send_action()`），三个阶段在同一线程内串行执行，任一阶段的性能瓶颈均会直接影响整体帧率。

在实际部署中，本文选用树莓派 5（ARM Cortex-A76，4 核，4 GB LPDDR4X）作为边缘推理平台，搭配 SO101 六自由度机械臂和 USB 摄像头，在无 GPU 的条件下运行 ACT 策略网络。初步测试表明，单次 ACT 推理耗时约 1500-3000 ms，而传感器采集和指令下发各仅约 1-15 ms，推理阶段占循环总时间的 99% 以上。在 chunk_size=100（即每次推理输出 100 帧动作）的配置下，系统实际帧率仅约 18–19 fps，距离 30 fps 的实时目标仍有明显差距。

在开源硬件与低成本本体方面，多个平台为机器人学习研究提供了可复现的硬件基础。LeRobot 配套的 SO-101 六自由度机械臂将硬件设计与学习框架深度绑定，结构件可 3D 打印，成本低廉，是本文实验平台的直接来源；ROBOTIS 的 OpenManipulator 系列[8] 提供配套 ROS2 的开源机械臂，硬件图纸与代码全开源，适合教学和原型开发；TurtleBot 系列[9] 作为开源移动机器人的事实标准，最新版本与 ROS2 生态深度集成；UC Berkeley HybridRobotics 实验室开发的 Berkeley Humanoid Lite[10]

采用 3D 打印方案，为小型人形机器人的开源化提供了新思路。

上述性能瓶颈涉及多个系统层次：PyTorch 推理引擎的算子调度、OpenMP 线程池的并行效率、Linux 内核的系统调用路径以及 USB/串口的硬件 I/O 延迟。然而，现有的机器人系统性能分析工作多聚焦于单一层次（如模型压缩、内核调度或通信延迟），缺乏从 AI 框架层到内核层的跨层次联合剖析，难以回答”瓶颈究竟在哪一层、由什么原因导致、各优化方向的收益几何”等关键问题。

因此，本文的研究意义在于：(1) 理论层面——建立覆盖 AI 推理框架层、Python 运行时层、OS 内核层和硬件 I/O 层的四层性能分析模型，提出面向”感知—推理—执行”紧耦合场景的多工具联合剖析方法；(2) 实践层面——端到端量化 LeRobot 在嵌入式 ARM 平台上的各阶段时间开销，定位主要性能瓶颈，系统评估 INT8 量化、异步推理流水线等优化方向的可行性与预期收益，为同类嵌入式机器人系统的性能优化与软件架构设计提供实验依据和工程参考。

1.2 研究目标

1. 建立跨层次（AI 框架层 / 运行时层 / OS 内核层 / 硬件 I/O 层）性能分析模型；
2. 端到端量化感知、推理、执行各阶段性能开销；
3. 定位主要瓶颈，为系统优化与软件架构改进提供实验依据。

1.3 主要贡献

- 提出覆盖 AI 框架到内核的四层性能分析模型；
- 通过 ftrace/torch.profiler/perf 多工具联合剖析，量化各阶段时间开销；
- 揭示 strace 对系统调用测量带来的 9× 时间放大问题，验证 ftrace 的必要性；
- 基于量化结果，系统梳理优化路径及其成本收益。

第二章 相关工作

2.1 机器人操作系统与通信框架

ROS2：机器人操作系统（Robot Operating System 2）是 ROS 的下一代版本，以 DDS（Data Distribution Service）作为底层通信中间件，支持节点间发布-订阅与服务

调用。ROS2 引入了节点生命周期管理、QoS 策略（可靠性/持久性/实时性配置）等企业级特性，支持跨机器跨平台分布式部署，并可通过 `rclcpp` / `rclpy` 接入 C++ 或 Python 节点。其通信延迟在局域网内通常为毫秒级，但节点调度仍依赖 Linux CFS 调度器，无硬实时保证。ROS2 主要运行于 Ubuntu（官方支持 22.04/24.04），需配合 Linux 内核使用，不支持 Windows/macOS 原生部署。

LCM（Lightweight Communications and Marshalling）：由 MIT 开发的高性能消息传递库，设计目标是为实时系统提供低延迟、零拷贝的消息序列化与传输能力。LCM 使用 UDP 单播/广播而非 TCP，支持用户态消息队列（类型声明 + 字节流），省去了 IDL 编译步骤，延迟可低至数微秒。但 LCM 仅提供消息传递层，不含控制回路、参数服务器或 RViz 等生态工具，生态远不如 ROS2。

[来源：LeRobot 软件栈全链路分析.md]

2.2 开源机器人硬件平台

SO-101：HuggingFace LeRobot 配套的六自由度机械臂，结构件可 3D 打印，成本低廉，与 LeRobot 框架深度绑定，是本文实验平台的直接来源。

OpenManipulator：ROBOTIS 推出的开源机械臂，提供配套的 ROS2 集成包，硬件图纸与代码全开源，适合教学和原型开发，支持 MoveIt! 运动规划。

TurtleBot4：开源移动机器人的事实标准，最新版本与 ROS2 生态深度集成，配备 OAK-D 深度相机，适用于室内导航与移动操作任务。

Berkeley Humanoid Lite：UC Berkeley HybridRobotics 实验室开发的 3D 打印小型人形机器人，为开源人形机器人平台提供了新思路。

2.3 机器人策略模型

机器人策略模型分为两大类：

视觉-语言-动作模型（VLA）：将视觉感知、语言理解和动作执行统一到端到端网络中，通过大规模预训练实现开放场景泛化能力。代表工作包括：

- **RT-2**：Google DeepMind 提出，将视觉-语言模型（VLM）输出直接映射为机器人动作 token；
- **OpenVLA**：开源 VLA 模型，支持多任务泛化，参数量 7B，可在消费级 GPU 部署；

- $\boldsymbol{\pi_0}$：Physical Intelligence 提出，以流匹配（Flow Matching）为核心动作生成机制，在双臂协作任务上表现优异。

模仿学习策略：通过专家演示数据训练策略网络，不依赖大规模预训练。代表工作包括：

- **ACT**（Action Chunking with Transformers）：CVAE 编解码器 + Transformer 动作预测，支持动作分块（chunk_size）和 time ensemble，适合桌面精细操作任务；
- **Diffusion Policy**：将动作生成建模为扩散过程，适合多模态动作分布。

对比：VLA 泛化能力强但计算量巨大（7B+ 参数），难以在嵌入式 ARM 平台实时运行；ACT/Diffusion Policy 参数量较小（ResNet18 + Transformer，约 50M 参数），可在树莓派 5 上运行，但推理延迟仍是主要瓶颈（2000–4000 ms/次）。

[来源：ACT 模型原理.md，ACT 推理.md]

2.4 嵌入式边缘 AI 推理

在资源受限的嵌入式平台上部署深度学习模型，需要针对硬件特性进行优化：

TFLite：Google 推出的移动端推理框架，支持 FP16/INT8 量化、ARM NEON SIMD 加速，广泛用于边缘设备推理。

ONNX Runtime：微软推出的跨平台推理引擎，支持多种执行提供者（CPU/CUDA / TensorRT），在 ARM 上通过 OpenMP 和 NEON 加速，延迟优于 TFLite 约 1.5–2×。

PyTorch Mobile：PyTorch 的移动端部署方案，支持 TorchScript 导出和 CPU 优化，但相比 ONNX Runtime 在 ARM 上的优化程度较低。

树莓派 5（Cortex-A76）计算能力：4 核 4 GB LPDDR4X，双发射 SIMD 单元支持 NEON，矩阵乘法（GEMM）理论峰值约 50 GFLOPS（FP32），实际推理 ACT 模型约 30–50 GFLOPS 利用率，实测推理耗时 2000–4000 ms/次。作为对比，RT-2（7B 参数）在树莓派 5 上几乎无法运行，VLA 模型需在 GPU 平台上部署。

2.5 系统级 Profiling 技术

机器人系统的性能分析需要跨层次工具链：

- **strace**：通过 `ptrace` 拦截系统调用，开销极大（9× 时间放大），仅用于定性探索；
- **ftrace**：内核态 tracepoint，开销极低（纳秒级），适合生产环境测量；
- **perf**：硬件性能计数器（PMC），可测量 IPC、Cache-miss 等微架构指标；
- **torch.profiler**：PyTorch 算子级 profiling，分解前向/反向传播各算子耗时。

已有机器人系统性能分析工作多聚焦于单一层次（模型压缩、内核调度或通信延迟），缺乏从 AI 框架层到内核层的跨层次联合剖析。

[来源：内核数据搜集分析.md，方法论.md]

第三章　系统架构与负载特征分析

3.1　LeRobot 软件栈架构

五层结构：

层级	主要组件
应用层	record.py / RecordRunner / lerobot-* CLI
AI 框架层	ACT policy / ResNet18 / Transformer
运行时层	PyTorch / ATen / OpenMP 线程池 / OpenCV
内核层	futex / pselect6 / ioctl / V4L2 / TTY
硬件层	USB 摄像头 / Feetech SCServo / ttyACM0

关键代码入口:`lerobot/src/lerobot/record.py`,核心循环:`get_observation()` → `select_action()` → `send_action()`。

[来源：LeRobot 软件栈全链路分析.md]

3.2　三类典型工作负载特征

三类任务：**抓取 (Grasp / pick)**、**推倒 (Push)**、**分类 (Sort / classification)**。统一硬件（树莓派 5）、统一模型架构（ACT，cs=100）、统一目标帧率（30 fps），每类任务采集 $n = 20$ 个 episode，episode 时长统一对齐至约 30 s 以排除时长差异干扰。

指标	抓取	推倒	分类
Episode 均值时长（s）	30.0	30.7	30.0
obs_pct（%）	15.4 ± 1.6	15.4 ± 1.9	16.7 ± 1.9
inference_pct（%）	36.5 ± 2.5	34.9 ± 1.2	35.9 ± 2.0
action_pct（%）	4.8 ± 0.3	4.4 ± 0.3	4.4 ± 0.3
wait_pct（%）	43.3 ± 2.3	**45.2** ± 2.0	42.9 ± 2.2
摄像头路数	1	1	1
关节自由度	6	6	6

[来源：实验数据/02_ 三类任务工作负载对比/cs100_task_comparison.md]

3.3 感知—推理—执行三阶段划分

- 阶段边界（代码级）：`get_observation()` / `select_action()` / `send_action()`；
- 时间预算目标：30 fps ⇒ 33.3 ms/帧；
- 当前实测（cs=100）：fps ≈ 18–19，推理占 ≈ 37%，等待占 ≈ 42%。

第四章　跨层次性能分析模型与剖析方法

4.1 四层性能分析模型

层级	可观测指标	工具
AI 推理框架层	算子耗时、张量形状、FLOP	torch.profiler
Python 运行时层	GIL 争用、帧循环时间	Python logging
OS 内核层	系统调用次数/耗时、线程调度	ftrace, strace
硬件 I/O 层	传输字节数、等待时延	perf, ioctl trace

4.2 工具链选型

strace vs ftrace 机制与开销对比：

两者作用层次不同，开销来源也不同：

- **strace**：通过 `ptrace` 系统调用在用户态拦截目标进程的每一次系统调用入口与出口。每次拦截都需要一次进程上下文切换（目标进程 → strace 进程），导致系

统时间从 **2.20 s 放大至 20.12 s（约 9×）**。其开销与系统调用频率正相关，频繁调用时干扰极大，不可用于生产级测量。

- **ftrace**：通过 Linux 内核 tracepoint / `trace_printk()` 在内核态直接写入 ring buffer，**插桩本身开销极低**（纳秒级，无进程切换）。但将 ring buffer 内容导出至用户态文件（`cat /sys/kernel/debug/tracing/trace > log`）是一次批量文件 I/O，当 event 密集时，**导出过程本身会消耗额外 CPU 和内存带宽**，且导出期间 ring buffer 可能覆盖旧数据（丢失事件）。
- **本文策略**：以 ftrace 插桩数据为主要内核态度量依据；strace 仅用于初期定性探索系统调用分布，不计入最终时序数据。

[来源：内核数据搜集分析.md，方法论.md]

4.3 实验配置

- 硬件：树莓派 5（ARM Cortex-A76，4 核，8 GB LPDDR4X）；
- 软件：LeRobot v0.3.4 / PyTorch 2.x / Python 3.10 / Ubuntu 24.04；
- 关键参数：chunk_size=100，fps=30，串口波特率 =1 Mbps；
- 每类任务采集 ≥ 3 次 episode 取均值，排除冷启动影响。

第五章　感知阶段性能剖析

5.1 时间剖析

5.1.1 摄像头路径

调用链：`cv2.VideoCapture.read()` → `grabFrame`（VIDIOC_DQBUF）→ `retrieveFrame`（libjpeg 解码）。

- pselect6 行为：摄像头后台线程阻塞 **23–32 ms**，符合 30 fps 帧间隔物理极限；
- MJPEG 解码耗时：**10–30 ms/帧**（libjpeg CPU 密集，是摄像头路径主要开销）；
- 整体摄像头贡献观测耗时约 **1.9 ms**（主线程视角，后台异步）。

[来源：pselect6 最终结果.md，OpenCV 视频流数据流分析.md]

5.1.2 舵机路径

调用链:`motors_bus.sync_read()` → `GroupSyncRead` → `PortHandler` → `/dev/ttyACM0`。

- Sync Read 包格式：`[0xFF 0xFF 0xFE LEN 0x82 ADDR LEN` $ID_1..ID_6$ `CHK]`；
- Present_Position 寄存器地址 56，sign-magnitude 编码（bit15 为方向位）；
- 耗时：**1.2 ms/帧**，开销可忽略。

[来源：observation 读取 ACM0 数据完整流程.md，Lerobot 内舵机读写完整链路.md]

5.2 CallStack 源码分析

5.2.1 摄像头路径调用链

- **用户态入口**：`cv2.VideoCapture.read()`（Python/OpenCV）
- **OpenCV 内部**:`VideoCapture_dispatch()` → `cv::CVCapture_CAM_v4l::grabFrame()`
- **系统调用**：`ioctl(VIDIOC_DQBUF)` → 从 V4L2 驱动取帧
- **后台线程**：`pselect6()` 阻塞等待新帧（23–32 ms）
- **解码路径**：`retrieveFrame()` → `cv::jpeg::decode()`（libjpeg，CPU 密集）
- **内核路径**：`read()` → `vfs_read()` → `USB` 摄像头驱动

关键性能结论：`pselect6` 在内核态的等待本身符合物理极限（30 fps ≈ 33.3 ms），但 libjpeg CPU 解码（10–30 ms）是主要可优化点。

[来源：OpenCV 视频流数据流分析.md，pselect6 最终结果.md]

5.2.2 舵机路径调用链

- **用户态入口**：`motors_bus.sync_read()`（scservo_sdk）
- **SDK 层**：`GroupSyncRead::txPacket()` → `PortHandler::writePort()`
- **系统调用**：`write(fd, buf, len)` → `sys_write()` → TTY 层
- **内核路径**：`tty_write()` → `usb_bulk_msg()` → `/dev/ttyACM0`
- **回包路径**：`read()` → `tty_read()` → `usb_bulk_msg()` → 用户态

- **协议解析**：`GroupSyncRead::rxPacket()` 解析返回位置数据

关键性能结论：`write` + `read` 组合耗时约 1.2 ms，开销可忽略；瓶颈不在系统调用路径，而在总线半双工串行协议本身。

[来源：Lerobot 内舵机读写完整链路.md，USB 串口写路径.md]

第六章　推理阶段性能剖析

6.1　时间剖析

6.1.1　chunk_size 参数扫描

chunk_size	FPS	inference_pct	wait_pct
1	0.8	99.7%	≈0%
10	5.7	85%	13%
50	15.1	48%	36%
100	18.5	37%	42%

FPS 与 cs 呈非线性：cs 从 1→10 增益巨大（0.8→6），cs 从 50→100 趋缓（15→19），符合边际递减规律。推理绝对耗时接近恒定（≈1–1.5 s/次），分块是”分摊”而非”减少”推理开销。

6.1.2　三类任务推理时间对比

在统一 cs=100、统一 episode 时长（≈30 s）的条件下，三类任务（抓取/推倒/分类）各采集 $n = 20$ 个 episode，仅看时间占比百分比以屏蔽 episode 总时长的干扰：

任务	inference_pct	wait_pct	action_pct
抓取（pick）	36.5% ± 2.5%	43.3% ± 2.3%	4.8% ± 0.3%
推倒（push）	34.9% ± 1.2%	**45.2%** ± 2.0%	4.4% ± 0.3%
分类（classification）	35.9% ± 2.0%	42.9% ± 2.2%	4.4% ± 0.3%

核心结论：

- **推理占比三者趋同（34.9%–36.5%，差距 < 2pp）**——在 cs 一致、模型结构一致时，单次推理时延与场景视觉复杂度无关，验证了”ACT 推理延迟由模型结构决定”的假设。

- **推倒任务 wait 占比最高（45.2%）**——因推/压动作行程更大、执行时间更长，系统空等比例上升，瓶颈出现在执行侧而非推理侧；优化方向为减小 cs（如 cs=50）以缩短每次 chunk 等待时长。
- 早期初步实验中分类任务 inference_pct 偏高（39.6%）的现象，被定位为 **60 s 长 episode 导致树莓派 CPU 热降频**，在统一为 30 s 后该差异消失，与模型本身负载无关。

[来源：实验数据/02_ 三类任务工作负载对比/cs100_task_comparison.md]

6.1.3 算子级耗时分解

[新实验：torch.profiler op-level profiling，分解 ResNet18 / Transformer Encoder / Decoder / 前后处理占比]

模块	耗时（ms）	占推理比例（%）
ResNet18 特征提取	**[TODO：]**	**[TODO：]**
CVAE Encoder（4 层 Transformer）	**[TODO：]**	**[TODO：]**
Action Decoder（1 层 Transformer）	**[TODO：]**	**[TODO：]**
前后处理（归一化、queue 写入）	**[TODO：]**	**[TODO：]**
合计	**[TODO：]**	100%

6.1.4 PyTorch 线程行为

- futex 调用：**64,569 次 / 40 s episode**，总等待时间 3,252 s（27 线程并行累计）；
- op=0（FUTEX_WAIT）占多数，op=9（FUTEX_WAIT_BITSET）用于 ATen 线程池；
- ARM Cortex-A76 使用 OpenMP 并行，NEON SIMD 加速 GEMM，但计算量仍是瓶颈。

[来源：futex 最终结果.md，ACT 推理.md]

6.2 CallStack 源码分析

6.2.1 ACT 推理完整调用链

- **Python 入口：`select_action()`**（RecordRunner 类）

- **ACT Policy**:`policy.select_action(observation)` → `Model.forward(image, state)`
- **ResNet18 特征提取**:`torchvision.models.resnet18()` → `Conv2d / BatchNorm / ReLU` 算子
- **CVAE Encoder**:4 层 Transformer Encoder → `torch.nn.functional.multi_head_attentio`
- **Action Decoder**: 1 层 Transformer Decoder → 动作分布采样
- **PyTorch 运行时**: `torch::jit::Graph` → `ATen` → `OpenMP parallel_for`
- **GEMM 底层**: `OpenBLAS / NEON SIMD` 矩阵乘法（ARM Cortex-A76 优化）
- **系统调用**: `futex()` 线程同步（64,569 次 / 40 s episode）

关键性能结论: 推理瓶颈在用户态 GEMM 计算（ARM CPU 矩阵乘法），与视觉输入复杂度无关；`futex` 系统调用本身开销极低（<1 s），但 OpenMP 线程池内 27 线程并行累计等待时间达 3,252 s，说明线程同步竞争显著。

[来源: ACT 推理.md, futex 最终结果.md]

第七章　执行阶段性能剖析

7.1　时间剖析

7.1.1　sync_write 路径

调用链:`motors_bus.sync_write()` → `GroupSyncWrite.txPacket()` → `write()` → TTY → USB bulk transfer。

- 无回包，半双工 TTY 总线，耗时 **1–3 ms**;
- 开销可忽略，不是系统瓶颈。

[来源: USB 串口写路径.md, motors_bus 异步读写设计分析.md]

7.2 CallStack 源码分析

7.2.1 舵机写路径完整调用链

- **用户态入口**：`motors_bus.sync_write(position_dict)`（scservo_sdk）
- **SDK 层**：`GroupSyncWrite::txPacket()` 组包 → `PortHandler::writePort()`
- **协议层**：半双工 TTY 总线，`write()` 调用后等待 1–3 ms 完成
- **系统调用**：`write(fd, buf, len)` → `sys_write()` → `tty_write()`
- **内核层**：`usb_bulk_msg()` → `dwc_otg` USB 驱动 → 物理 USB 传输
- **半双工约束**：总线冲突保护，同一时刻仅允许一个方向传输

关键性能结论：`write` 系统调用本身耗时仅 38 s，执行阶段瓶颈不在软件路径，在机械臂执行本身（关节行程、物理限幅）。

[来源：USB 串口写路径.md，Lerobot 内舵机读写完整链路.md]

第八章 异步化可行性调研

8.1 摄像头异步读

LeRobot 摄像头采集通过 OpenCV 后台线程实现异步：主线程调用 `retrieveFrame()` 时，后台线程已在 `pselect6()` 等待下一帧。

- **当前状态**：已是默认实现，无需改造；
- **可优化点**：MJPEG 解码（10–30 ms/帧）为 CPU 密集瓶颈，切换为 YUYV 格式可直接消除解码开销；
- **结论**：摄像头异步读已实现，当前主要矛盾在解码格式而非异步化。

8.2 推理异步

LeRobot 支持双进程推理架构（主进程 + 推理进程，通过 gRPC 通信）：

- **当前状态**：已实现但未启用（当前配置下 stall 不可避免）；

- **不 stall 条件推导**：

$$N \geq 2 \times T_{\text{infer}} \times F$$

代入参数（$T_{\text{infer}} = 2\,\text{s}$，$F = 30\,\text{fps}$，$N = 100$）：

$$\text{需要 } N \geq 120, \quad \text{当前 } N = 100 < 120 \quad \Rightarrow \text{stall 不可避免}$$

- **结论**：配合 INT8 量化（预期 T_{infer} 降至 0.5–1 s）后，可将不 stall 所需 N 降至 30–60，当前 $N = 100$ 可满足条件，推理异步方可启用。

[来源：async_inference_ 推理执行流水线分析.md]

8.3 舵机异步读

舵机位置读取通过 `sync_read` 实现（半双工写请求 + 读回包），异步化改造存在以下难点：

- **总线半双工约束**：读写操作必须串行，异步并发写入会导致总线冲突；
- **安全限幅链路**：`send_action()` 需先读取当前位置（read-modify-write），若读操作异步化，则无法保证限幅判断的时效性；
- **结论**：在当前半双工总线协议下，舵机异步读难以实现，不建议作为优化方向。

[来源：Lerobot 内舵机读写完整链路.md，motors_bus 异步读写设计分析.md]

8.4 舵机异步写

类似异步读，舵机指令下发（`sync_write`）的异步化存在根本性约束：

- **总线半双工冲突**：异步并发写会导致总线冲突，指令帧损坏；
- **安全限幅链路断裂**：`send_action()` 中的 read-modify-write 操作依赖同步读返回的实时位置，若写操作先于读完成，限幅判断将基于过期数据，可能导致机械臂碰撞；
- **相关工作**：调研现有机器人控制系统异步执行架构[**async_robot**]，均在支持全双工或具备独立控制通道的硬件平台上实现，不适用于半双工 TTY 总线；
- **结论**：舵机异步写在当前硬件平台上不可行，亦非必要（写操作本身仅 1–3 ms）。

[来源：motors_bus 异步读写设计分析.md]

第九章　性能瓶颈分析与优化建议

9.1　瓶颈归因

- **主瓶颈**：ACT 推理（ARM CPU 矩阵乘法），2000–4000 ms/次；
- **次瓶颈**：MJPEG 解码（10–30 ms/帧，可替换为 YUYV 消除）；
- **非瓶颈确认**：系统调用本身（pselect6 421 ns，futex <1 μs，均远低于推理耗时）。

9.2　跨阶段综合分析

9.2.1　时间预算表（单帧，33.3 ms 目标）

阶段	实测耗时	占目标比例
感知（舵机 + 图像，主线程视角）	5.2 ms	15.6%
推理（chunk 内均摊）	**[TODO：]** ms	**[TODO：]**%
执行（sync_write）	4.8 ms	14.4%
其余调度开销	**[TODO：]** ms	**[TODO：]**%
实际单帧循环（推理帧）	2200–4200 ms	**66×–126× 超时**

9.2.2　系统调用分布（40 s episode）

系统调用	次数	均值耗时	主要来源
futex	64,569	<1 μs	PyTorch 线程池
pselect6	36,191	421 ns	摄像头后台线程 V4L2 等待
write	35,651	38 μs	USB 串口写

结论：系统调用本身开销极低，瓶颈在用户态 CPU 推理计算，非 IO。*[来源：recordloop 每部分时间全部分析.md，内核数据搜集分析.md]*

9.3 优化方向分析

优化方向	预期收益	可行性	复杂度
INT8/FP16 量化	推理降 2–4×	高	中
ONNX Runtime 后端	推理降 1.5–2×	高	低
MJPEG → YUYV	解码降至 0	高	低
推理异步（gRPC 双进程）	消除 stall（需先完成量化）	中	高
异步舵机读	不可行	×	—
异步舵机写	不可行	×	—

9.4 软件架构改进方向

- 推理模块解耦（独立进程 + gRPC），避免 GIL 与主循环相互干扰；
- 摄像头格式统一为 YUYV，消除 libjpeg CPU 解码路径；
- 面向机器人负载的内核优化方向：实时调度器（SCHED_FIFO）、USB 驱动优先级提升。

第十章 总结与展望

10.1 主要工作总结

1. 建立了覆盖 AI 框架到内核的四层性能分析模型；
2. 通过 Python 日志 + ftrace + torch.profiler 多工具联合，端到端量化了各阶段开销；
3. 揭示 ACT 推理（2000–4000 ms）为主瓶颈，系统调用 overhead 可忽略；
4. 推导了异步推理流水线不 stall 的数学条件，指出当前参数配置的局限性；
5. 系统梳理了各优化路径的可行性与成本收益；
6. 对感知、推理、执行三阶段进行了从 Python 到 kernel 的全链路 CallStack 源码分析；

7. 调研了各阶段异步化可行性，论证了舵机异步读/写的不可行性及其原因。

10.2 研究局限性

- 仅在 SO101 单臂场景下测试，未覆盖双臂或移动机器人；
- 未实施量化等优化方案并验证，结论停留在分析层面；
- 无 GPU 对比基线，难以量化 CPU 推理与加速器的差距。

10.3 未来工作

- INT8 量化 + ONNX Runtime 实验，验证推理加速收益；
- 扩展至多任务场景（更多抓取类别）与多机器人平台；
- 探索面向机器人负载特征的实时操作系统设计。

参考文献

[1] BROHAN A, BROWN N, CARBAJAL J, et al. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control[J]. arXiv preprint arXiv:2307.15818, 2023.

[2] KIM M J, PUMACAY K, JATAVALLABHULA K M, et al. OpenVLA: An Open-Source Vision-Language-Action Model[J]. arXiv preprint arXiv:2406.09246, 2024.

[3] CADENE R, ALIBERT S, et al. LeRobot[EB/OL]. 2024. https://github.com/huggingface/lerobot.

[4] ZHAO T Z, KUMAR V, LEVINE S, et al. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware[C]//Proceedings of Robotics: Science and Systems (RSS). 2023.

[5] CHI C, FENG S, DU Y, et al. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion[C]//Proceedings of Robotics: Science and Systems (RSS). 2023.

[6] INTELLIGENCE P.
pi0: A Generalist Robot Policy[EB/OL]. 2024. https://www.physicalintelligence.com/.

[7] ZHAN T, et al. Mobile ALOHA: Bimanual Mobile Manipulation with Low-Cost Whole-Body Teleoperation[C]//IEEE International Conference on Robotics and Automation (ICRA). 2024.

[8] ROBOTIS. OpenManipulator[EB/OL]. 2024. https://emanual.robotis.com/docs/en/platform/openmanipulator_x/.

[9] ROBOTICS O, ROBOTICS C. TurtleBot 4[EB/OL]. 2023. https://www.turtlebot.com/.

[10] HybridRobotics. Berkeley Humanoid Lite[EB/OL]. 2025. https://lite.berkeley-humanoid.org/.

附　录

附录 A：待补充实验清单

优先级	实验内容	填充章节	预计耗时
★★★	推倒/分类任务计时数据	第 5.2.2 节三类任务对比	**已完成**
★★★	torch.profiler op-level	第 5.2.3 节算子分解	1 天
★★	CPU 频率锁定对比	第 5.4 节峰值成因	半天
★	perf stat（IPC, cache-miss）	判断 compute vs memory bound	半天

致　谢

[TODO：致谢内容]